

Self-Stabilizing Resource Allocation

Atomicity Refinement
Allowing Neighborhood Concurrency

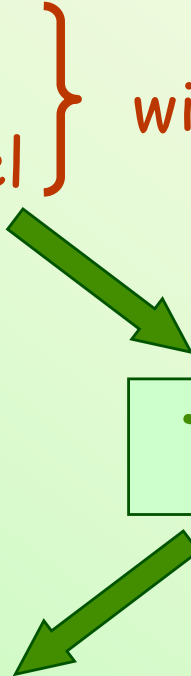
Atomicity and Daemon Refinement

[Nesterenko and Arora 1999]

High Atomicity Model

- ❖ Daemon
- ❖ Communication model

} with strong assumptions



TRANSFORMER

Low Atomicity Model

- ❖ Daemon
- ❖ Communication model

} with weak assumptions

Daemon Concurrency

Central



Distributed

Communication Model

State-Reading Model

Access to the state of all neighbors and update of state, atomically

Daemon Concurrency

Central



Distributed

Communication Model

State-Reading Model



Read/write atomicity Model

Atomic access to the state of any neighbor or atomic update of state

Atomicity and Daemon Refinement

[Nesterenko and Arora 1999]

High Atomicity Model

- ❖ Central Daemon
- ❖ State-Reading Model



TRANSFORMER

Low Atomicity Model

- ❖ Distributed Daemon
- ❖ R/W Atomicity Model

Atomicity and Daemon Refinement

Well-studied in self-stabilization area...

Atomicity and Daemon Refinement

- ❖ [Mizuno and Kakugawa, WDAG 1996]

A transformation of self-stabilizing programs for distributed computing environments

- ❖ Message Passing.
- ❖ Non-Silent.
- ❖ Unbounded Timestamps.

Atomicity and Daemon Refinement

- ❖ [Mizuno and Kakugawa, WDAG 1996]

- ❖ [Gouda and Haddix, WSS 1997]

 - The Linear Alternator*

 - ❖ Distributed Daemon/State Model.
 - ❖ Non-Silent.
 - ❖ 1 bit.
 - ❖ Only on arrays

Atomicity and Daemon Refinement

- ❖ [Mizuno and Kakugawa, WDAG 1996]

- ❖ [Gouda and Haddix, WSS 1997]

- ❖ [Mizuno and Nesterenko, IPL 1998]

A transformation of self-stabilizing serial model programs for asynchronous parallel computing environments

- ❖ Distributed Daemon/ R/W Atomicity Model.

- ❖ Silent.

- ❖ Unbounded timestamps.

Atomicity and Daemon Refinement

- ❖ [Mizuno and Kakugawa, WDAG 1996]
- ❖ [Gouda and Haddix, WSS 1997]
- ❖ [Mizuno and Nesterenko, IPL 1998]
- ❖ [Gouda and Haddix, WSS 1999]

The Alternator

- ❖ Distributed Daemon/State Model.
- ❖ Non-Silent.
- ❖ Bounded space - $O(\Delta)$.

Atomicity and Daemon Refinement

- ❖ [Mizuno and Kakugawa, WDAG 1996]
- ❖ [Gouda and Haddix, WSS 1997]
- ❖ [Mizuno and Nesterenko, IPL 1998]
- ❖ [Gouda and Haddix, WSS 1999]
- ❖ [Nesterenko and Arora, DISC 1999]
Stabilization-Preserving Atomicity Refinement
 - ❖ Distributed Daemon/ R/W Atomicity Model.
 - ❖ Silent.
 - ❖ Bounded space - $O(\Delta)$.

Atomicity and Daemon Refinement

- ❖ [Mizuno and Kakugawa, WDAG 1996]
- ❖ [Gouda and Haddix, WSS 1997]
- ❖ [Mizuno and Nesterenko, IPL 1998]
- ❖ [Gouda and Haddix, WSS 1999]
- ❖ [Nesterenko and Arora, DISC 1999]
- ❖ [Beauquier, Datta, Gradinariu, and Magniette, DISC 2000]
Self-stabilizing local mutual exclusion algorithm.
 - ❖ Distributed Daemon/ R/W Atomicity Model.
 - ❖ Non-Silent.
 - ❖ Bounded space - $O(n^2)$.

[Boulinier, Cantarell, Petit, and Villain, tech.rep. 2003]

<http://www.laria.u-picardie.fr/~petit/publi/noteBLME.ps.gz>

Atomicity and Daemon Refinement

- ❖ [Mizuno and Kakugawa, WDAG 1996]
- ❖ [Gouda and Haddix, WSS 1997]
- ❖ [Mizuno and Nesterenko, IPL 1998]
- ❖ [Gouda and Haddix, WSS 1999]
- ❖ [Nesterenko and Arora, DISC 1999]
- ❖ [Beauquier, Datta, Gradinariu, and Magniette, DISC 2000]

[Boulinier, Cantarell, Petit, and Villain, tech.rep. 2003]

<http://www.laria.u-picardie.fr/~petit/publi/noteBLME.ps.gz>

- ❖ [Kakugawa and Yamashita, SRDS 2002]

Self-stabilizing local mutual exclusion on networks in which process identifiers are not distincts.

- ❖ Distributed Daemon/ R/W Atomicity Model.
- ❖ Non-Silent.
- ❖ Bounded space - $O(n)$.

Atomicity and Daemon Refinement

- ❖ [Mizuno and Kakugawa, WDAG 1996]
- ❖ [Gouda and Haddix, WSS 1997]
- ❖ [Mizuno and Nesterenko, IPL 1998]
- ❖ [Gouda and Haddix, WSS 1999]
- ❖ [Nesterenko and Arora, DISC 1999]
- ❖ [Beauquier, Datta, Gradinariu, and Magniette, DISC 2000]

[Boulinier, Cantarell, Petit, and Villain, tech

<http://www.laria.u-pic>

ME.ps.gz

- ❖ [Kakugawa et al.,

**Based on a Local Mutual
Exclusion Mechanism**

LME Problem

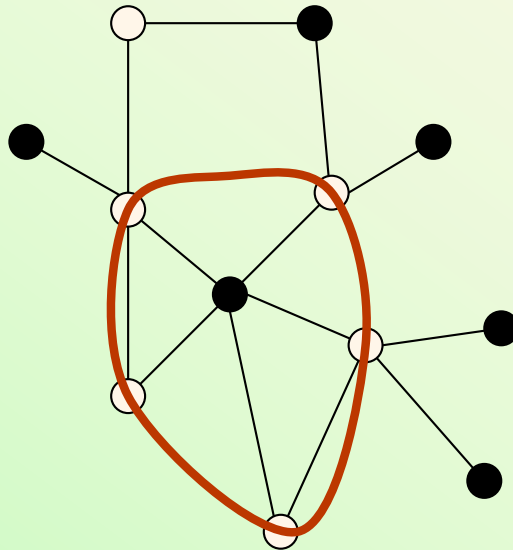
❖ Safety

If a process is executing its critical section, then none of its neighbors is executing its critical section simultaneously.

❖ Liveness

If a process requests to enter its critical section, then it will eventually execute its critical section.

LME Algorithm

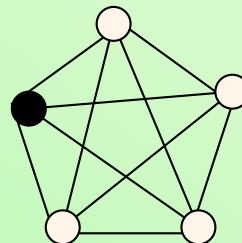


Concurrency

Best (expected) case



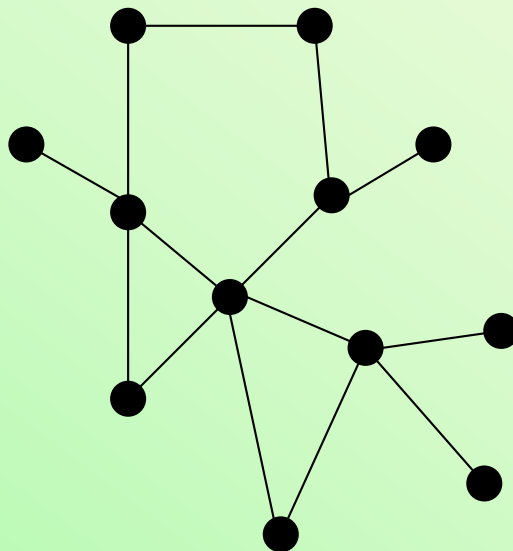
Worst case



i.e., Mutual Exclusion

Question

Could we increase concurrency?



Answer

Yes!

Local Resource Allocation !

Local Resource Allocation

- ❖ Statement

Neighboring Processes share m resources with compatibility criteria

- ❖ Compatibility

Two resources X and Y are said to be compatible, denoted $X \leftrightarrow Y$, if two neighboring processes can access X and Y concurrently

Local Resource Allocation

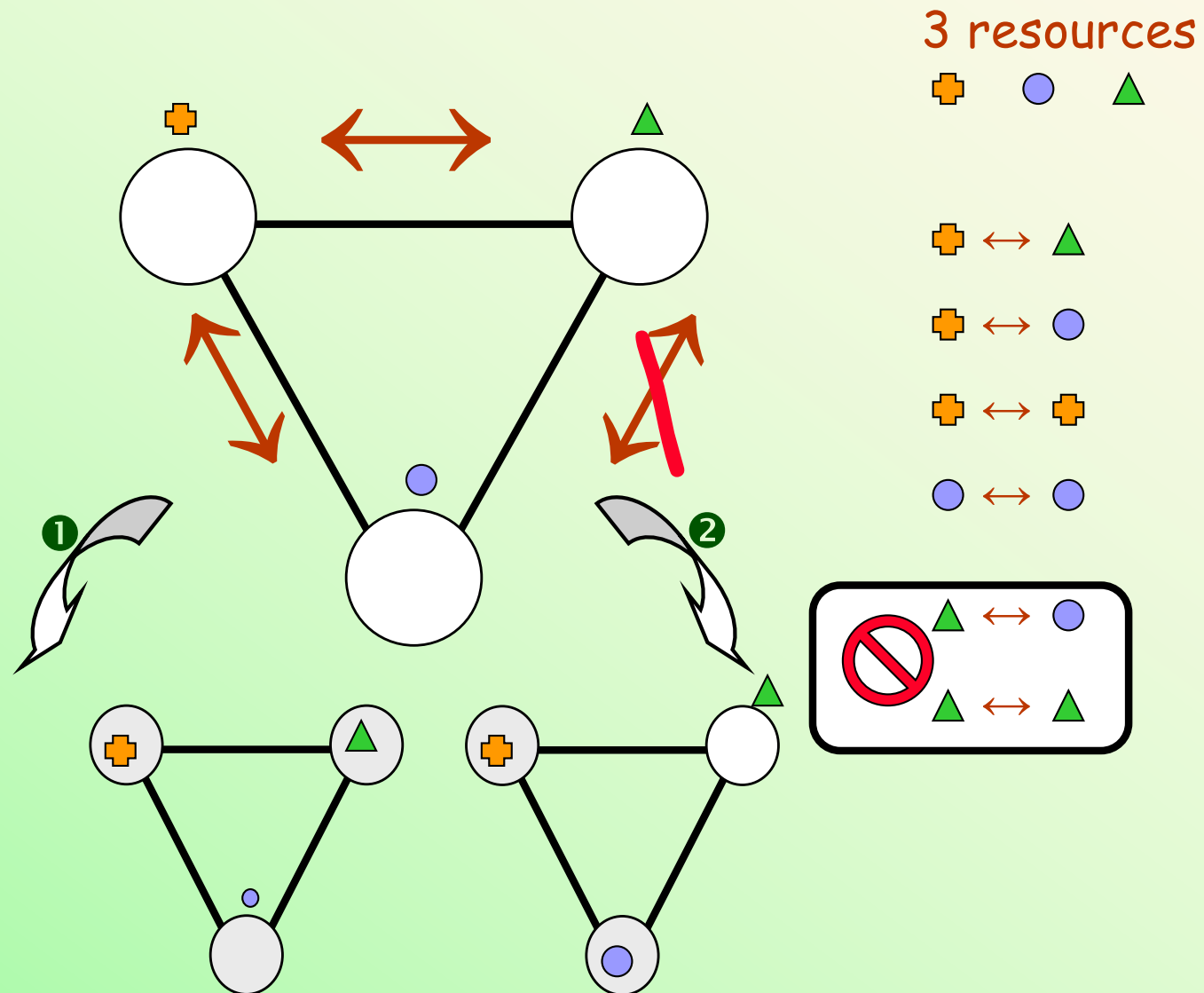
❖ Safety

If two neighboring processes p and q are executing their critical section concurrently using X and Y , respectively, then $X \leftrightarrow Y$.

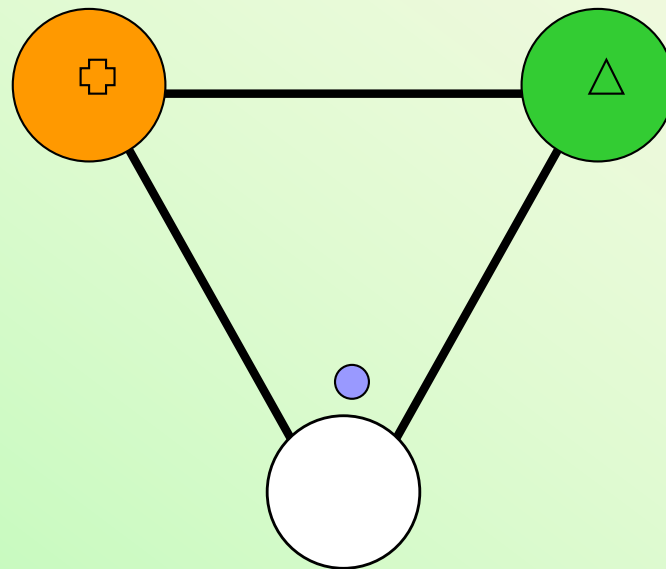
❖ Liveness

If a process requests to enter its critical section, then it will eventually execute its critical section.

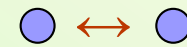
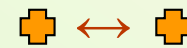
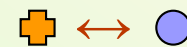
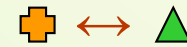
Local Resource Allocation



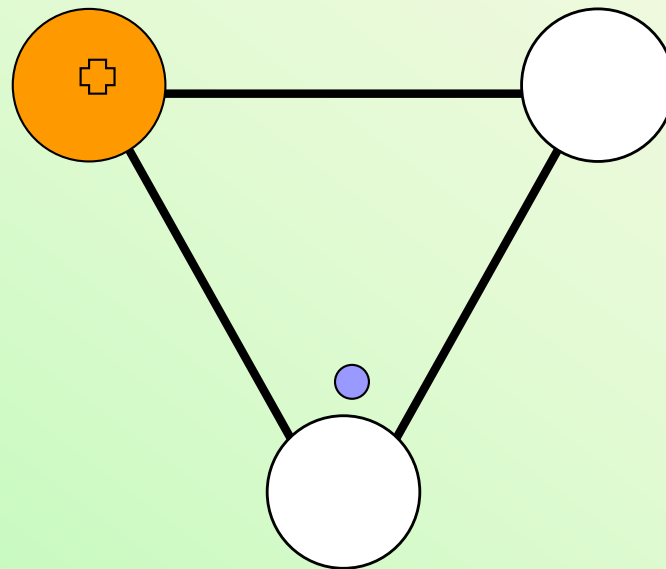
Local Resource Allocation



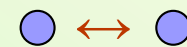
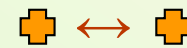
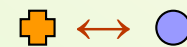
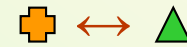
3 resources



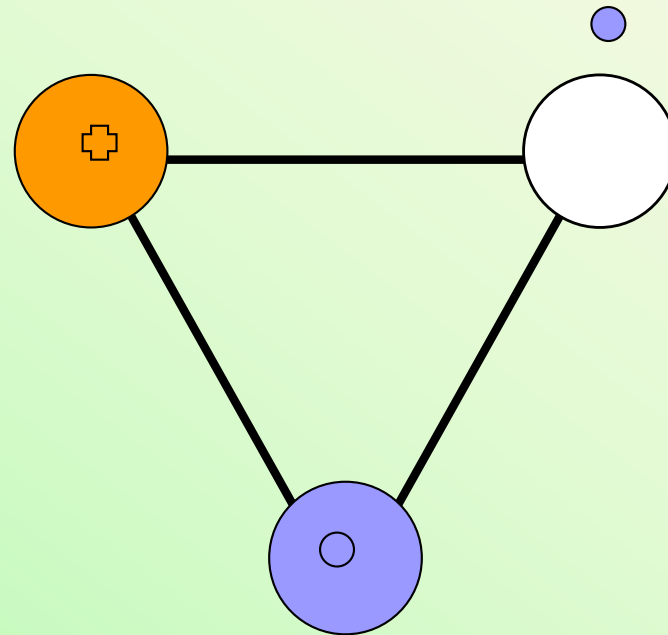
Local Resource Allocation



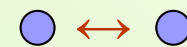
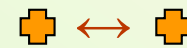
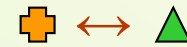
3 resources



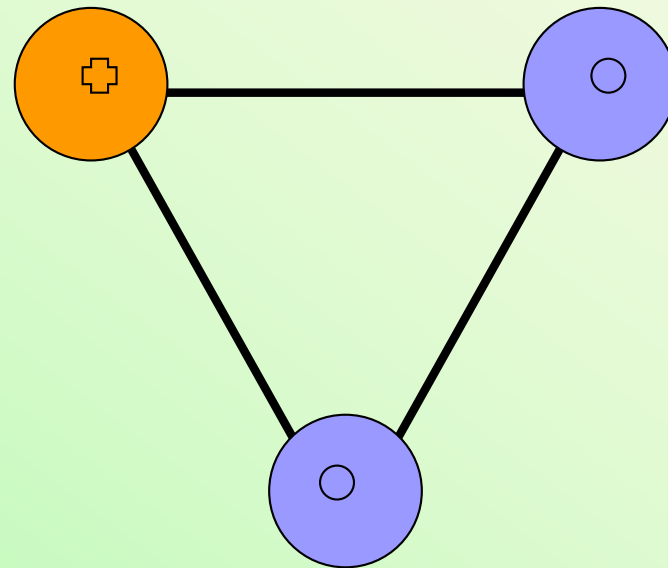
Local Resource Allocation



3 resources



Local Resource Allocation



3 resources

+ • ▲

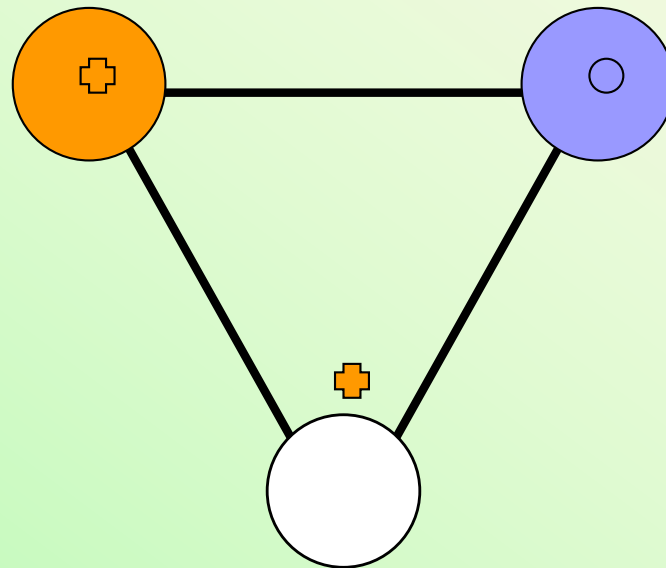
+ ↔ ▲

+ ↔ •

+ ↔ +

• ↔ •

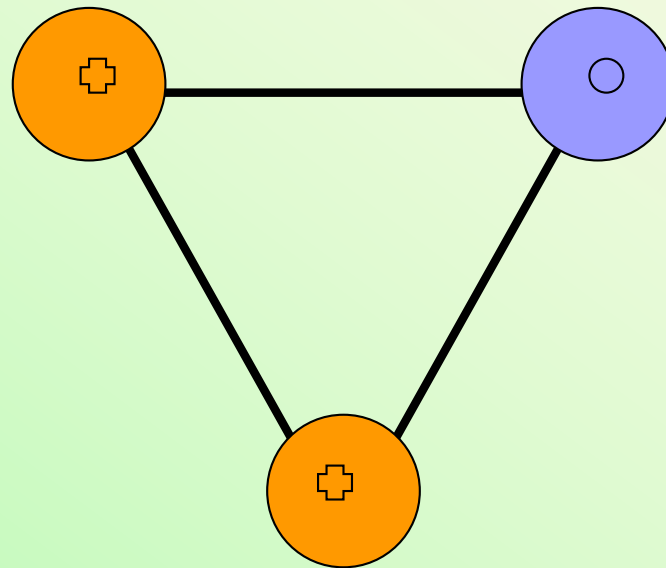
Local Resource Allocation



3 resources



Local Resource Allocation



3 resources

⊕ ● ▲

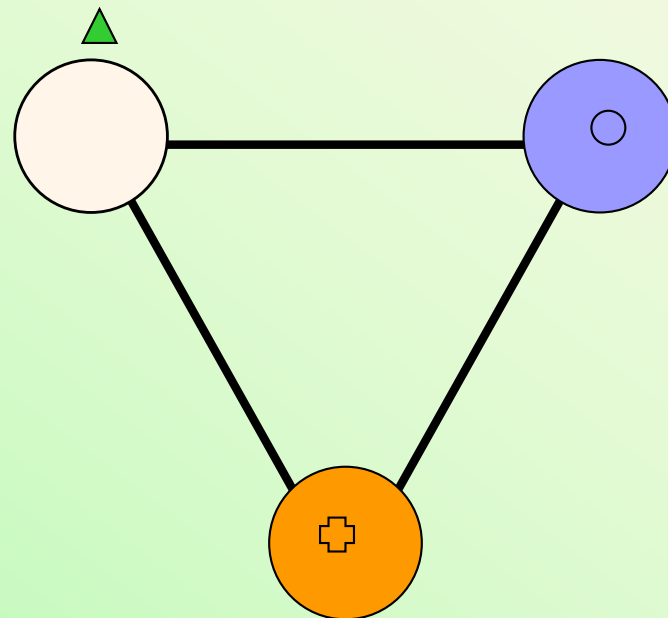
⊕ ↔ ▲

⊕ ↔ ●

⊕ ↔ ⊕

● ↔ ●

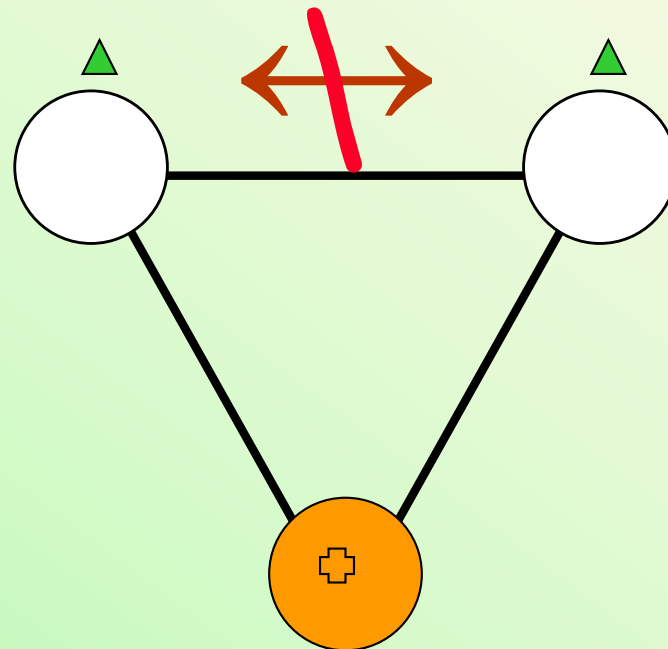
Local Resource Allocation



3 resources



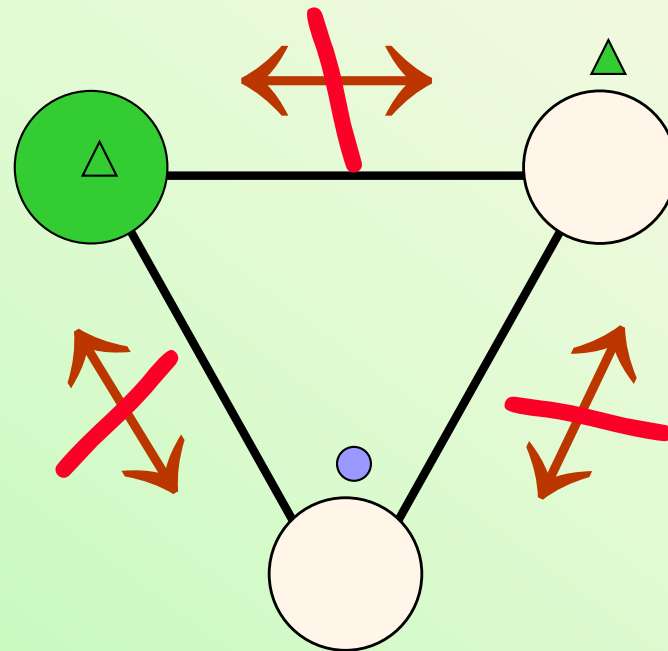
Local Resource Allocation



3 resources



Local Resource Allocation



3 resources

⊕ ● ▲

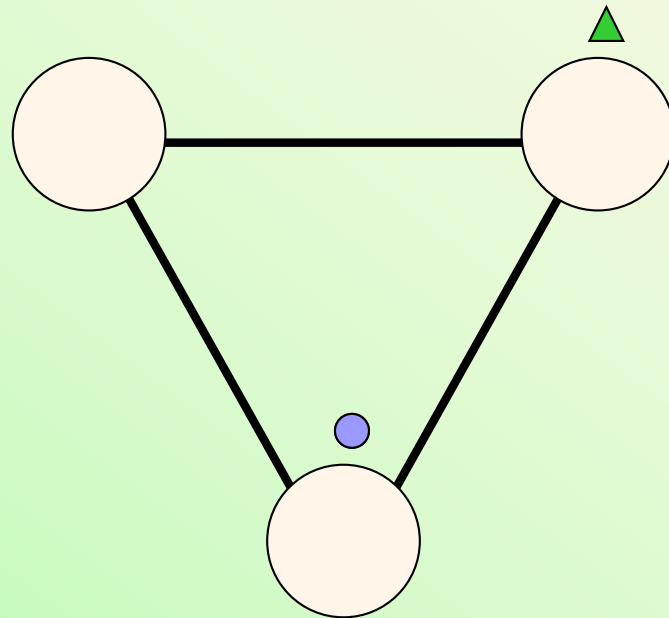
⊕ ↔ ▲

⊕ ↔ ●

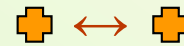
⊕ ↔ ⊕

● ↔ ●

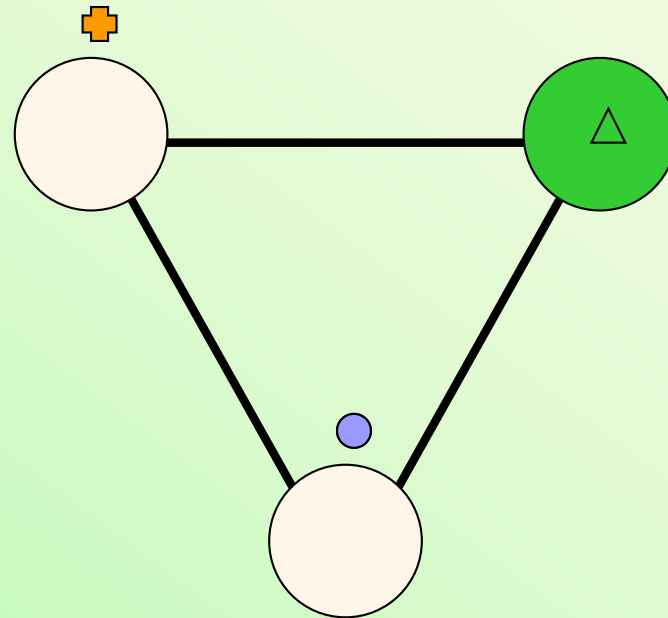
Local Resource Allocation



3 resources



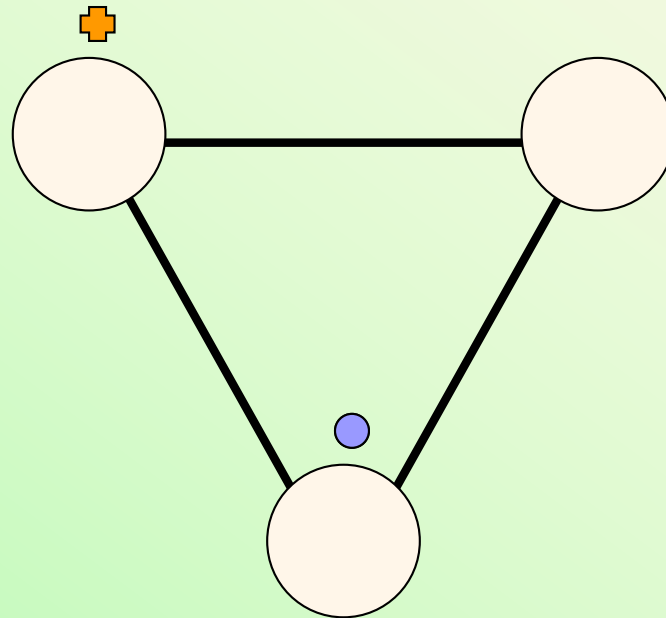
Local Resource Allocation



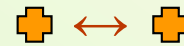
3 resources



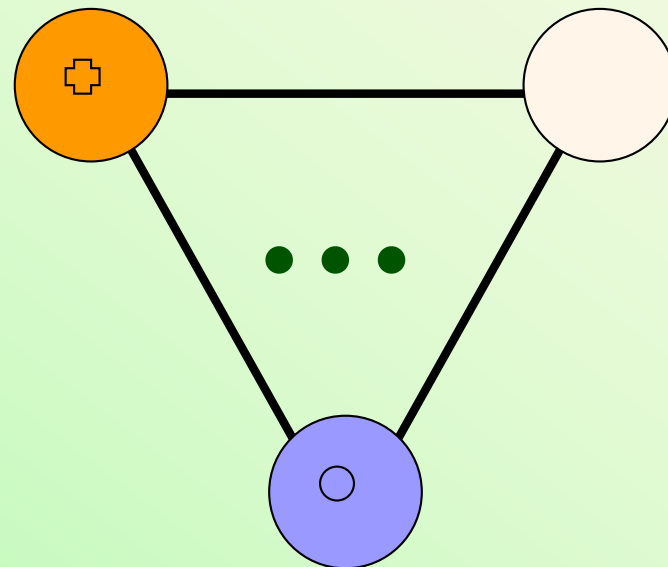
Local Resource Allocation



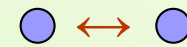
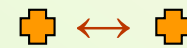
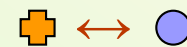
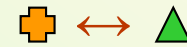
3 resources



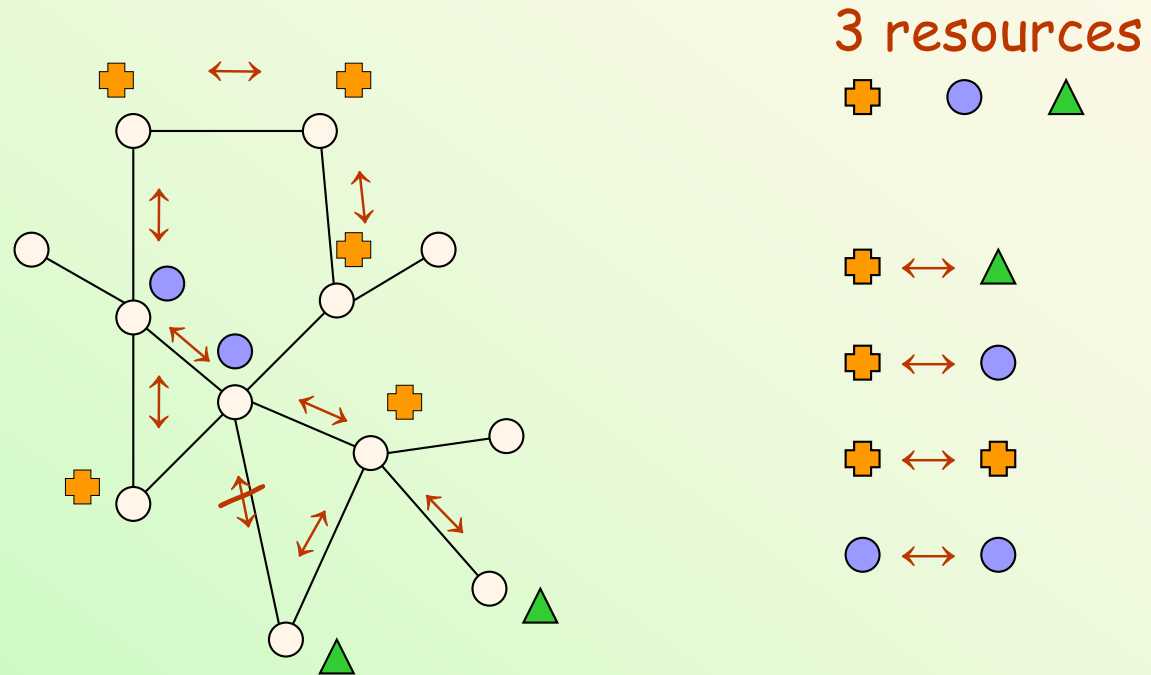
Local Resource Allocation



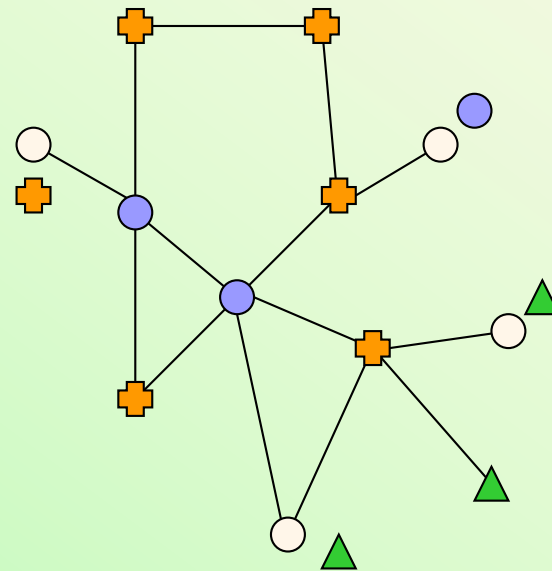
3 resources



Local Resource Allocation



Local Resource Allocation



3 resources

✚ ● ▲

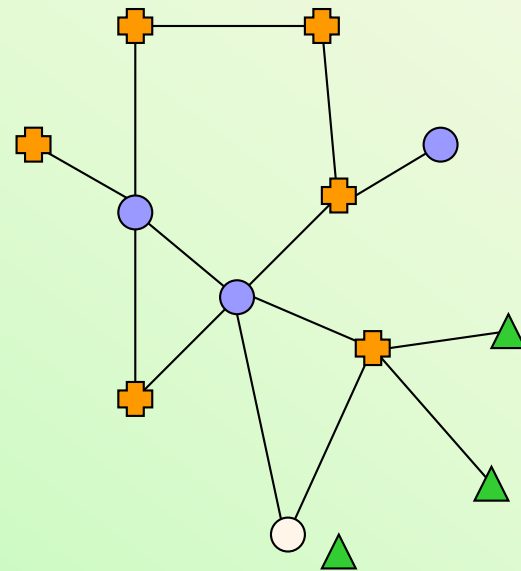
✚ ↔ ▲

✚ ↔ ●

✚ ↔ ✚

● ↔ ●

Local Resource Allocation



3 resources

⊕ ● ▲

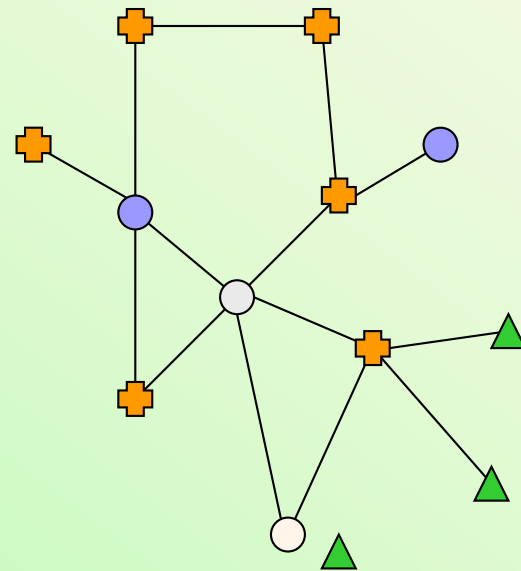
⊕ ↔ ▲

⊕ ↔ ●

⊕ ↔ ⊕

● ↔ ●

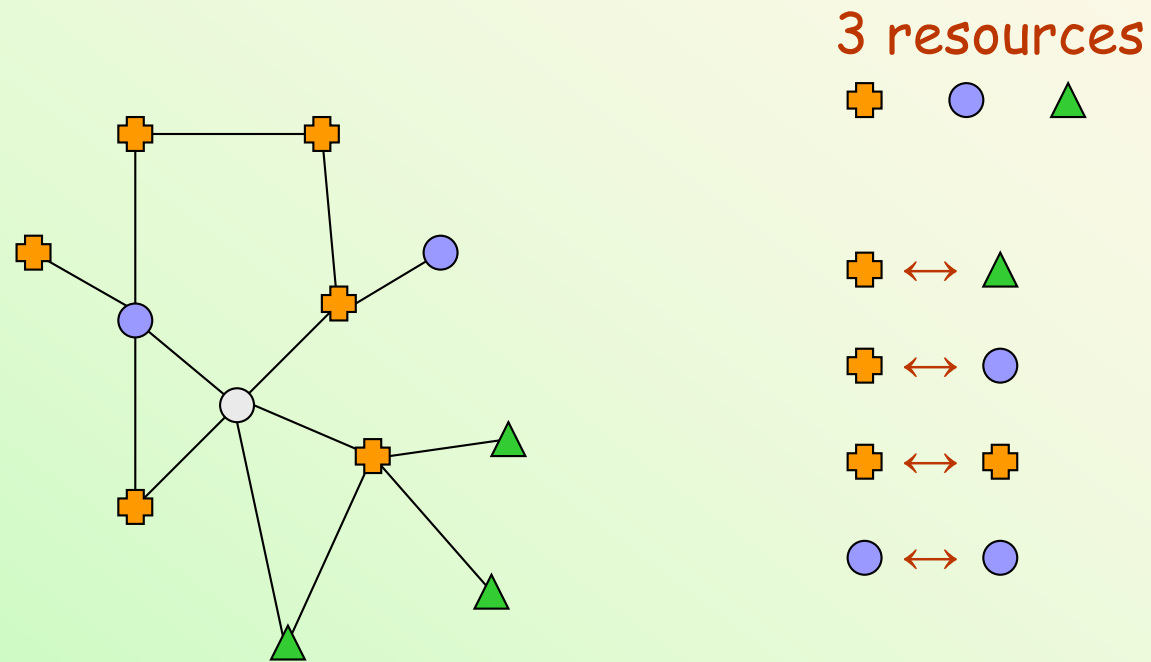
Local Resource Allocation



3 resources



Local Resource Allocation



Relation With Other Resource Allocation Problems?

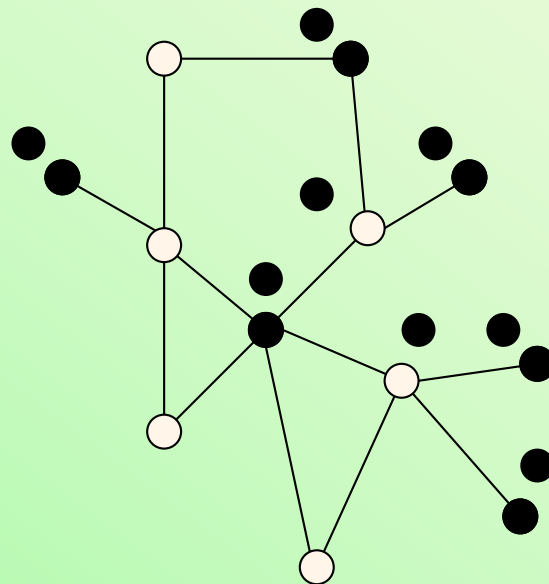
Related Resource Sharing Problem

- ❖ Dining Philosophers [Dijkstra, EWD 625, 1978]
(Local Mutual Exclusion)

One Resource Shared in Exclusive Access by Neighbor Processes

Concurrency = 1 process in the neighborhood

LRA solves LME



1 resource only



Related Resource Sharing Problem

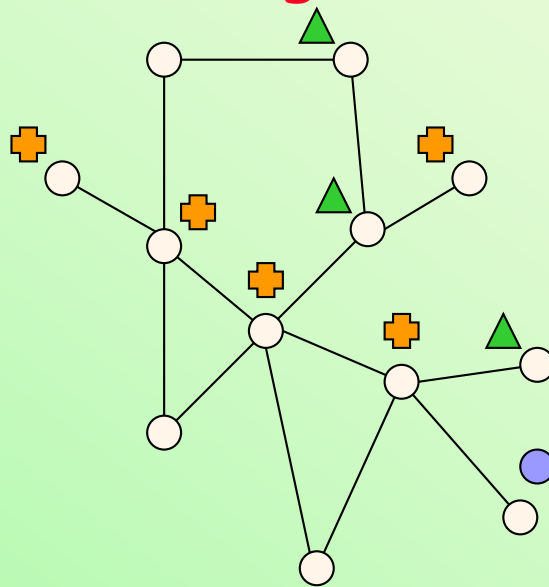
❖ Drinking Philosophers [Chandy and Misra, TOPLAS 1984]

Several Resources Shared in Exclusive Access by Neighbor Processes

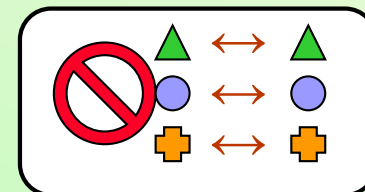
Concurrency = 1 process in the neighborhood for each resource

Neighbors are allowed to use resources simultaneously provided that they are using different resources

LRA solves Drinking Philosophers Problem



3 resources



Related Resource Sharing Problem

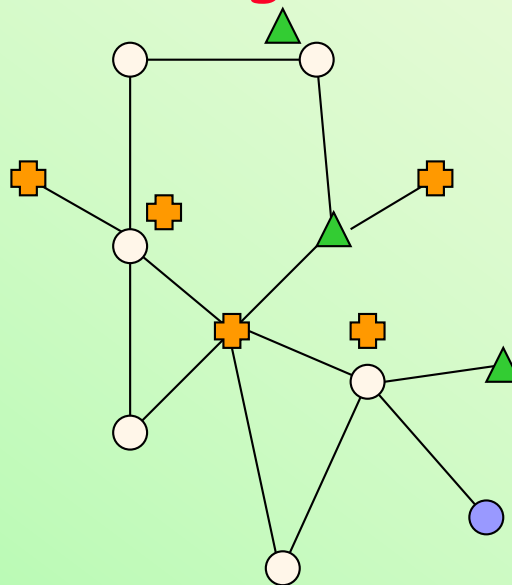
❖ Drinking Philosophers [Chandy and Misra, TOPLAS 1984]

Several Resources Shared in Exclusive Access by Neighbor Processes

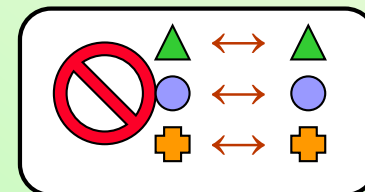
Concurrency = 1 process in the neighborhood for each resource

Neighbors are allowed to use resources simultaneously provided that they are using different resources

LRA solves Drinking Philosophers Problem



3 resources



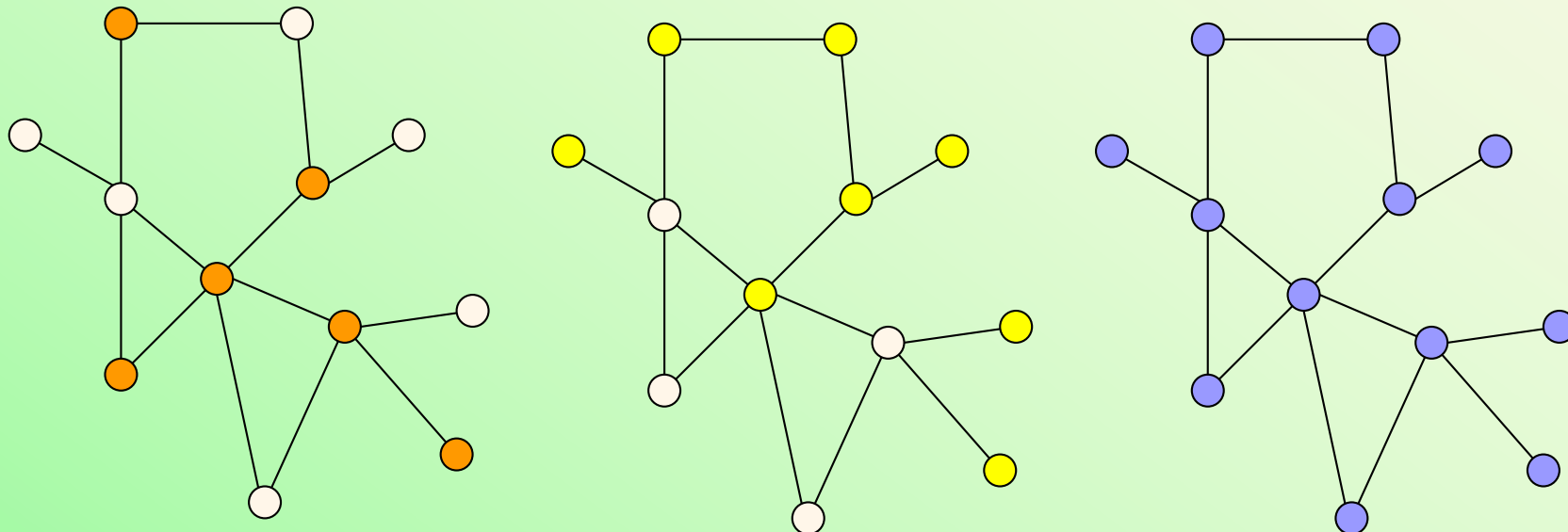
Related Resource Sharing Problem

❖ Group Mutual Exclusion [Joung, DISC 1998]

M Exclusive Resources Shared in Concurrent Access

Concurrency = n processes

... but exclusive resource access



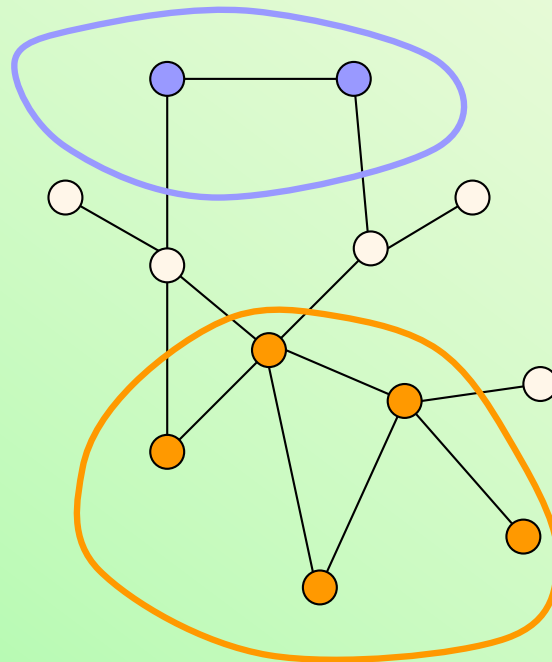
Related Resource Sharing Problem

❖ Local Group Mutual Exclusion

M Local Exclusive Resources Shared in Concurrent Access

Concurrency = $\Delta+1$ processes in the neighborhood

... but exclusive resource access



Related Resource Sharing Problem

❖ Local Group Mutual Exclusion

M Local Exclusive Resources Shared in Concurrent Access

Concurrency = $\Delta+1$ processes in the neighborhood

... but exclusive resource access

Does LRA solves the (Local) GME Problem?

Yes !

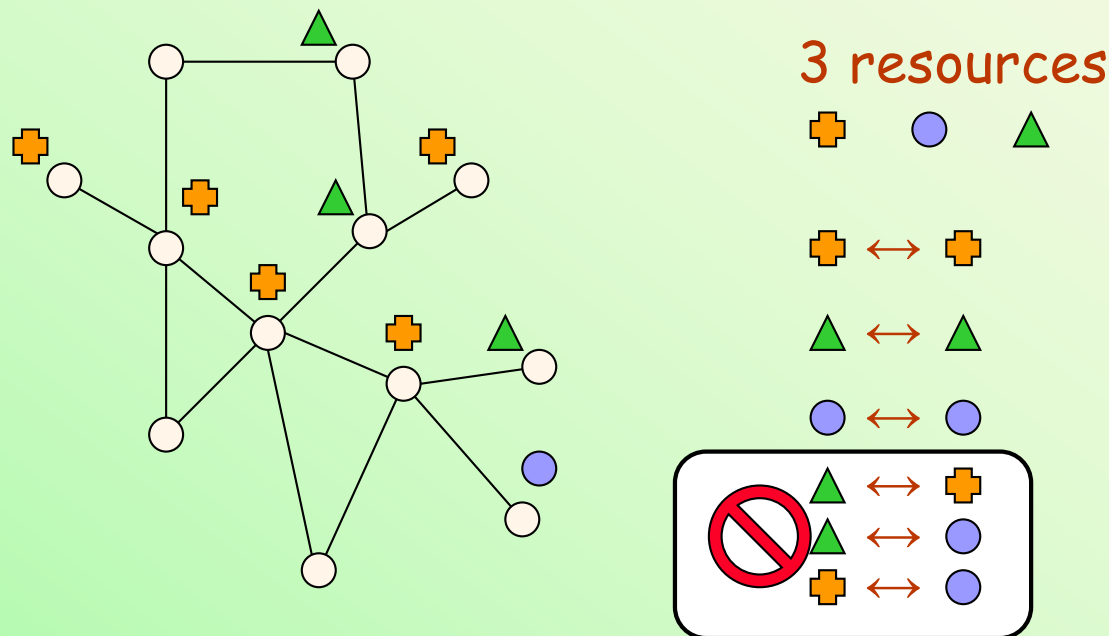
Related Resource Sharing Problem

❖ Local Group Mutual Exclusion

M Local Exclusive Resources Shared in Concurrent Access

Concurrency = $\Delta+1$ processes in the neighborhood

... but exclusive resource access



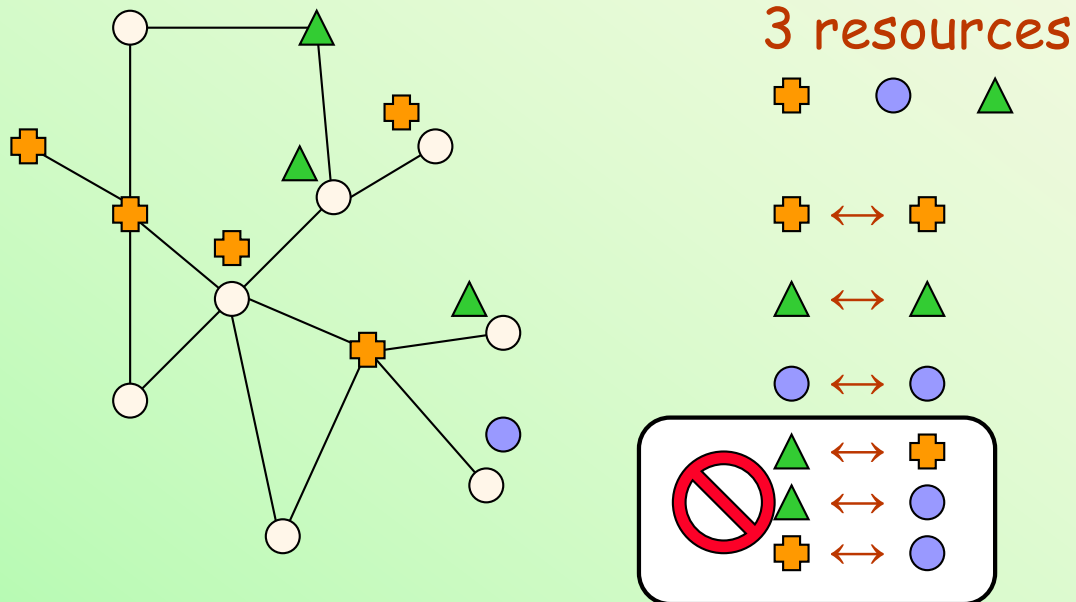
Related Resource Sharing Problem

❖ Local Group Mutual Exclusion

M Local Exclusive Resources Shared in Concurrent Access

Concurrency = $\Delta+1$ processes in the neighborhood

... but exclusive resource access



Related Resource Sharing Problem

LRA is a generalisation of

- ❖ Local Mutual Exclusion
- ❖ Dining and Drinking Philosophers
- ❖ Local Group Mutual Exclusion
(and then Local Readers/Writers)

An SS LRA Algorithm

Each process p maintains

- ❖ A compatibility graph $CG = (R, C)$
 - R = Resources accessed by p
 - C = Compatibility Edges
- ❖ *Request* $\in R \cup \{\perp\}$ (upper layer)
- ❖ *Grant*: Boolean (LRA layer)
- ❖ c : integer, a Lamport's timestamp (LRA layer)

An SS LRA Algorithm

Entry Section

- ❖ Each time a process p requests to access the critical section with Resource X (Upper Layer):
 - p sets Request to X (Upper Layer)
 - p updates its timestamp c (LRA Layer)
- ❖ Then, p waits for one of the two following conditions to become true (Grant to true by the LRA Layer):
 - p has the maximum timestamp in its neighborhood, or
 - Request (X) is compatible with its neighbors

Exit Section

- ❖ p resets Request to $\perp$, Grant to false, and c to 0 (LRA Layer)

The Upper Layer Algorithm

•
•
•

REQ : $(\text{Request} = \perp) \wedge \neg \text{Grant} \wedge (X \in R \text{ requested}) \rightarrow \text{Request} := X$

CS : $(\text{Request} \neq \perp) \wedge \text{Grant} \rightarrow \langle\langle \text{CS} \rangle\rangle; \text{Request} := \perp$

•
•
•

No request $\Rightarrow$ No action $\Rightarrow$ Silent Algorithm

The LRA Layer Algorithm

$$p \triangleleft q \equiv (c < c_q) \vee ((c = c_q) \wedge (p < q))$$

$$\text{Ready} \equiv (\forall q \in N: (c_q > 0) \Rightarrow ((p \triangleleft q) \vee ((\text{Request}_q \neq \perp) \Rightarrow (\text{Request} \leftrightarrow \text{Request}_q))))$$

$$A1 : (\text{Request} \neq \perp) \wedge \neg \text{Grant} \wedge (c = 0) \rightarrow c := \max \{ c_q \in N \} + 1$$

$$A2 : (\text{Request} \neq \perp) \wedge \neg \text{Grant} \wedge (c > 0) \wedge \text{Ready} \rightarrow \text{Grant} := \text{true}$$

$$A3 : (\text{Request} = \perp) \wedge (\text{Grant} \vee (c > 0)) \rightarrow \text{Grant} := \text{false}; c := 0$$

Self-Stabilizing LRA Scheme

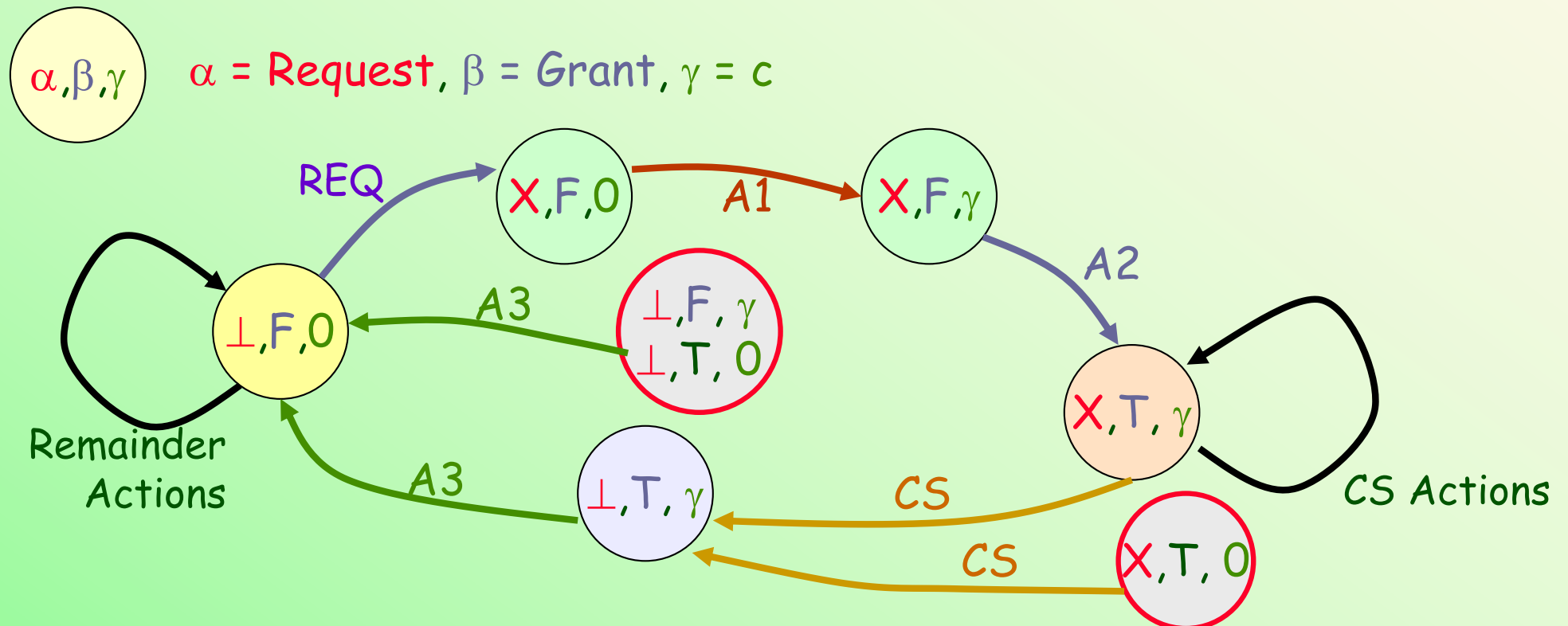
REQ : $(\text{Request} = \perp) \wedge \neg \text{Grant} \wedge (X \in R \text{ requested}) \rightarrow \text{Request} := X$

CS : $(\text{Request} \neq \perp) \wedge \text{Grant} \rightarrow \ll \text{CS} \gg; \text{Request} := \perp$

A1 : $(\text{Request} \neq \perp) \wedge \neg \text{Grant} \wedge (c = 0) \rightarrow c := \max \{ c_q \in N \} + 1$

A2 : $(\text{Request} \neq \perp) \wedge \neg \text{Grant} \wedge (c > 0) \wedge \text{Ready} \rightarrow \text{Grant} := \text{true}$

A3 : $(\text{Request} = \perp) \wedge (\text{Grant} \vee (c > 0)) \rightarrow \text{Grant} := \text{false}; c := 0$



Sketch of Proof

Same approach as in [Mizuno and Nesterenko and 1998]

1. Stabilize w.r.t.

$T \equiv [$ If $c_p > 0$ and Ready_p ,
then if a neighbor q changes c_q ,
then $c_p < c_q \Rightarrow c_p = 0$.
]

2. $T \Rightarrow \text{Safety}$

3. $T \Rightarrow \text{Liveness}$

Atomicity Transformation

❖ Action-Compatibility

Two actions A_p and A_q are said to be compatible, denoted $A_p \leftrightarrow A_q$,

if none of the variables written in the statement of Action A_p (resp. A_q) is read in Action A_q (resp. A_p).

❖ A compatibility graph $AG = (A, C)$

A = Action Labels of p and the Action Labels of all p 's neighbors

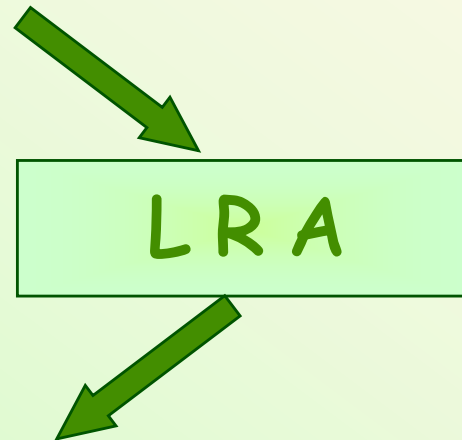
C = Action-Compatibility Edges

Atomicity Transformation

High Atomicity Program P

- ❖ Collection of actions of the form:

$$A_X : G_X \rightarrow S_X \quad (0 \leq X \leq k)$$



Low Atomicity Program P_+

- ❖ Variables of both P and LRA
- ❖ Actions of LRA algorithm
- ❖ For each A_X of P :

$$A^1_X : (\text{Request} = \perp) \wedge \neg \text{Grant} \wedge G_X \rightarrow \text{Request} := X$$

$$A^2_X : (\text{Request} = X) \wedge \text{Grant} \rightarrow S_X; \text{Request} := \perp$$

Conclusion

- ❖ Local Resource Allocation (LRA)
- ❖ Self-stabilizing LRA Algorithm
- ❖ Self-Stabilizing Atomicity Transformer
- ❖ R/W Atomicity Model
- ❖ Silent-Preserving